

Chitransh - Infrastructure / SRE Intern Assignment

Candidate

Name: Chitransh Prasanna

Role: Infrastructure / SRE Intern

GitHub Repository: <https://github.com/Chitransh1011/zeotap-incident-management-system>

Deadline: 6 May 2026

Project

Mission-Critical Incident Management System (IMS)

The repository contains a running backend and frontend implementation for incident ingestion, processing, storage, alerting workflow, RCA enforcement, and dashboard operations.

How to Run

Docker Compose:

```
bash
docker compose up --build
```

Backend: <http://localhost:4000>

Frontend: <http://localhost:5173>

What Was Built

- Async signal ingestion API.
- Bounded in-memory queue for backpressure.
- Token-bucket rate limiter on ingestion.
- Debouncing by component ID within a 10 second window.
- Raw signal audit log for high-volume payloads.
- Structured work-item and RCA source of truth.
- Hot dashboard cache for active incident views.
- Timeseries aggregation sink.
- Alerting Strategy pattern for component severity/routing.
- State transition validation for OPEN -> INVESTIGATING -> RESOLVED -> CLOSED.
- Mandatory RCA before closing incidents.
- Automatic MTTR calculation from RCA start/end time.

- React dashboard with live feed, incident detail, raw signals, status actions, and RCA form.
- Health endpoint and throughput metrics.
- Retry logic and unit tests for RCA validation.

Non-Functional Enhancements

- Optional API-key security using INGEST_API_KEY and X-IMS-API-Key.
- Request body size limit using MAX_BODY_BYTES.
- Configurable CORS origin.
- Security headers for content-type, referrer policy, and no-store cache behavior.
- Mutex-protected work-item updates to avoid status/RCA race conditions.
- Dockerized backend and frontend.

Documentation Included

- README.md
- docs/BACKEND.md
- docs/API.md
- docs/FRONTEND.md
- docs/MCP_HOST_MONITORING.md
- docs/TESTING.md
- docs/REQUIREMENTS_TRACEABILITY.md
- docs/ASSIGNMENT_PLAN.md
- docs/GITHUB_SUBMISSION.md

Verification

Backend unit tests passed:

```
bash
cd backend
npm test
```

Frontend production build passed:

```
bash
cd frontend
npm install
npm run build
```

Backend API smoke test passed using sample failure events:

- Health endpoint returned OK.

- Sample signal ingestion accepted events.
- Active incidents returned with RDBMS as P0.

Sample Failure Data

samples/failure-events.json simulates an RDBMS outage followed by MCP host and cache failures.

Run:

```
bash
node samples/simulate-failure.js
```